Secure Embedded Water Tank Controller

Working Draft with Partial Results

Finley Burns
CYCS 3240
Prepared November 10, 2026

***Abstract*—I am developing a C17 water-tank controller for an emulated STM32F4 Cortex-M4. The project uses one level sensor, one inlet valve, and a serial interface for configuration. The current prototype implements threshold control and invalid-input handling. Windows host tests passed 69 independent control cases, 18 sequence steps, 23 parser files, and three command-line error checks. The firmware still uses a fixed sensor reading. Changing sensor input, periodic sampling, serial command parsing, HMAC authentication, and the tank feedback model remain to be implemented. This draft records the design and the results available so far. It is not the completed December final report.**

***Index Terms*—embedded systems, C, input validation, simulation, software testing**

I. Project Description

I chose a single water-storage tank because it gives the controller a clear job and makes incorrect behavior easy to observe. The inlet should open when the level is too low and close when the level is too high. The controller should reject invalid settings and continue operating while it receives operator commands.

The project began with Rust, but the course requires C or C++. I switched to C17 because I already know C and could keep the same Renode and STM32F4 setup. This makes explicit range checks, buffer limits, and error handling important. I kept the control decisions separate from the hardware code so the host tests and firmware can use the same function.

My goals are to implement repeatable tank control, reject invalid or unauthorized settings changes, keep control running during malformed serial input, and evaluate the behavior using files. I am not building physical hardware, a network dashboard, or a multi-tank system. The model does not attempt detailed fluid dynamics.

II. Research and Design

A. Control Rule

The controller uses two thresholds. It opens the inlet below the low threshold and closes it above the high threshold. At either threshold and throughout the interval, it keeps its previous state. This hysteresis avoids switching whenever a reading crosses a single target value.

Levels use basis points from 0 to 10000. The default low and high thresholds are 3000 and 7000. Valid settings satisfy $0 \leq low < high \leq 10000$. A reading outside the level range, an invalid sample flag, or invalid settings closes the valve and sets a fault. When a valid sample returns, the same control rule resumes. Recovery inside the interval keeps the valve closed because that was its fault state.

The proposed sampling period is 100 ms. The completed design also specifies a maximum sample age of 300 ms. Age checks and the periodic timer are not part of the fixed-reading firmware yet. I will test 50 ms and 500 ms sampling periods to show how the time between readings affects the response.

B. Hardware and Emulator

The existing firmware uses USART2 for serial output and PD12 to represent the inlet valve. It uses the STM32F4 register definitions described in RM0090 [1]. Renode supplies the emulated processor and peripherals, and its Robot Framework integration supports repeatable firmware tests [2].

For changing sensor tests, the design reserves a small shared record in emulated RAM. The test adapter will publish a level, validity flag, and timestamp before the firmware reads the sample. A sequence field will identify an incomplete update. This provides a controlled test input but does not test ADC conversion or electrical sensor behavior.

C. Serial Commands

The planned command format is `SET,low,high,tag`, ending with LF. The tag will authenticate the exact bytes before the comma that precedes the tag. HMAC-SHA256 produces 32 tag bytes, represented here as 64 lowercase hexadecimal characters. I plan to use Mbed TLS rather than implement a cryptographic primitive myself [3]. The selected release and its target configuration still need to be pinned and tested.

The parser will accept at most 128 bytes before LF, check each number before converting it to a smaller type, and validate both thresholds before changing either setting. The line buffer includes a terminator. A partial line will expire above 500 ms. The control loop will process at most 32 received bytes and one authentication attempt per cycle. These limits are design choices whose timing still needs measurement.

HMAC does not encrypt commands or prevent replay. An old valid command can still be accepted in this design. The supplied key is public test data. The host and emulator are trusted. CERT C guidance informs the planned checks for integers, arrays, strings, and file handling [4].

D. Tank Model

The planned model has a 100 L capacity and starts at 50 L. An open inlet supplies 2 L/s and normal demand removes 1 L/s. For each interval, the model uses

$$V_{n+1} = V_n + (u_n Q_{in} - Q_{out})\Delta t, \tag{1}$$

where u_n is the actual modeled valve state. I will keep that separate from the commanded state so a stuck-open valve can continue filling after the controller requests closure. The model will record an empty or overflowing tank before limiting the displayed volume to its physical range.

The C model will use checked integer microliters and retain fractional remainders. Python will coordinate the scenario files and Renode. This division keeps control and model calculations in C while using Python for tool execution and logs.

III. Implementation and Tools

A. Completed Code

The current source contains the shared controller, a CSV runner for independent cases, and a second runner for stateful sequences. The host reader rejects embedded NUL, overlong records, invalid Boolean fields, numeric overflow, missing fields, and trailing input. It opens files in binary mode so Windows does not treat Ctrl-Z as end-of-file. It handles LF and CRLF blank lines explicitly.

The firmware contains startup assembly, a linker script, C initialization, board setup, and one fixed 25% control decision. The expected low-level output is an open valve. This is a working prototype, not the changing tank simulation.

The portable host entry point uses Python 3.9 or newer and GCC or Clang. It builds local executables, runs inputs from files, checks their outputs and exit codes, and saves each run in a separate directory. No extra Python packages are needed for these tests. The optional firmware builder requires GNU Arm Embedded GCC.

B. Work Still Required

The remaining implementation includes the sensor record, SysTick scheduling, USART2 receive handling, bounded queue and line parser, HMAC verification, settings updates, tank model, and emulator adapter. I have written their interfaces and pseudocode in the comprehensive design. They have not been demonstrated by the current host results.

The Renode test environment also has separate dependencies. Native Windows Robot automation for the referenced Renode version uses Cygwin and Python, or the Linux route under WSL. The host-only tests do not require those tools.

IV. Testing and Samples

A. Executed Host Tests

The 15 original tests cover basic threshold behavior, state retention, invalid samples, and invalid settings. I added 54 cases around both thresholds and the supported endpoints. The 18-step sequence preserves actual state and checks recovery after faults. The parser tests use exact-byte files and expected exit codes.

A sample control row is

```
2500,1,3000,7000,0,1,0
```

The fields mean level, valid flag, low threshold, high threshold, previous valve state, expected valve state, and expected fault. This row starts with a closed valve at 25% and expects it to open without a fault. A sequence also includes a reset field so the runner can distinguish a new scenario from another step in the same scenario.

One parser file deliberately gives a wrong expected state. The runner must return 1 for that mismatch. Malformed files and invalid invocation return 2. Successful tests return 0. Checking failure cases prevents a runner that always reports success from appearing correct.

B. Planned Integration Tests

The normal tank test will run for 600 simulated seconds three times. The demand-change test will use 0.5 L/s after 200 seconds and 1.5 L/s after 400 seconds. Other scenarios cover demand above inlet capacity, a stuck-open valve, a 3600-second run, and different sampling periods. Sensor tests will check invalid flags, range errors, stale readings, and recovery.

Authentication tests will include RFC 4231 HMAC-SHA256 vectors, correct commands, modified messages, wrong keys, malformed tags, and authenticated but invalid threshold values [5]. A replay case will record the known absence of replay protection. Heavy serial input will test whether the control schedule continues while the parser rejects traffic.

These are test plans. The data files exist, but their presence does not mean the future firmware or model has passed them.

V. Results So Far

Table I summarizes the November 9 Windows host run. The compiler was MinGW-w64 GCC 16.1.0 with C17, `-O1`, debug symbols, and warnings treated as errors. The same package passed again from a fresh extraction whose path contained spaces and parentheses.

TABLE I
Executed Windows Host Checks

Check	Count	Result
Independent control cases	69	Pass
Sequence steps	18	Pass
Parser test files	23	Pass
Command-line error checks	3	Pass

The 23 parser files include four accepted inputs, 18 rejected inputs, and one deliberate expected-state mismatch. These counts describe different kinds of checks and should not be combined into one coverage percentage. The results support the tested control and file-handling behavior, but they do not establish firmware timing, authentication, or complete memory safety.

An undefined-behavior and bounds sanitizer build failed at link time because the Windows compiler installation could not

find `libubsan`. No sanitized executable ran. The original macOS handoff reported 15 control cases, eight input rejections, and one fixed-reading Renode test passing. That earlier result does not verify the expanded tests on macOS. Linux execution also remains unverified: WSL failed before a Linux process started because the required virtualization support was unavailable.

A. Goals Reached

The shared control function and repeatable file-based host tests are complete for the cases measured here. Cross-platform entry points are supplied, but only Windows has run the expanded suite. The fixed-reading emulator test establishes an earlier firmware baseline. The authenticated operator interface, changing tank feedback, sensor-age checks, and timing goals remain incomplete.

VI. User Guide

Extract the complete November 12 package before running it. Install Python 3.9 or newer and a C17 GCC or Clang compiler. From the extracted root, use the following command on Windows:

```
py -3 Scripts/run_tests.py -cc gcc
```

On macOS or Linux, use:

```
python3 Scripts/run_tests.py -cc cc
```

Use the installed Python command and pass the quoted full compiler path after `-cc` if needed. The script creates a new `Runs` folder and prints its location. Read `summary.json` and the saved stdout/stderr files for details. Recorded `Evidence` stays separate from new runs.

The optional `-sanitize undefined` or `-sanitize address` mode needs a compiler with the corresponding runtime. A failed diagnostic build is not silently replaced by a normal build. The firmware builder requires `arm-none-eabi-gcc`. The supplied Renode test currently checks only the fixed-reading firmware.

VII. What Must Be Added Before Submission

I need to implement the remaining modules and collect actual results for the tests in Section IV-B. The final version must identify the selected cryptographic library release, record the tool and firmware versions, include representative level/valve traces, measure control timing during serial traffic, and compare results against each stated goal. Any platform or test that remains unavailable must still be labeled accurately.

The final presentation must use the same verified results. Neither the draft slides nor this report should be presented as evidence that unfinished features work. The December package will need a fresh source inventory and a complete run from its own extracted folder.

VIII. Conclusion

The current C prototype makes the expected valve decisions in the executed host tests and rejects the malformed files included in the suite. Separating the control function from the board code has made these checks repeatable. The next step is to connect the scheduled firmware, authenticated commands, and changing tank model so I can evaluate the complete project rather than only its current foundation.

References

[1] STMicroelectronics, "RM0090 STM32F4 reference manual." [Online]. Available: https://www.st.com/resource/en/reference_manual/rm0090-stm32f4xx-reference-manual-stmicroelectronics.pdf

[2] Renode, "Testing with Renode." [Online]. Available: https://renode.readthedocs.io/en/latest/introduction/testing.html

[3] Mbed TLS project. [Online]. Available: https://github.com/Mbed-TLS/mbedtls

[4] SEI, "CERT C Coding Standard." [Online]. Available: https://cmu-sei.github.io/secure-coding-standards/sei-cert-c-coding-standard/

[5] M. Nystrom, "Identifiers and Test Vectors for HMAC-SHA-224, HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512," RFC 4231, Dec. 2005.